

CYBERSECURITY INTERNSHIP

Name: Hammad Rasheed

ID: DHC-5590

Week 4

Advanced Threat Detection & Web Security

Task 1

Intrusion Detection & Monitoring

Method of Fail2Ban Setup

Install Fail2Ban

```
sudo apt update && sudo apt install fail2ban -y
```

Create Jail Configuration:

```
sudo nano /etc/fail2ban/jail.local
```

Add configuration:

```
[juiceshop-login]
enabled = true
port = http,https
filter = juiceshop-login
logpath = /var/log/juiceshop/access.log
maxretry = 5
bantime = 3600
findtime = 600
```

Create Filter:

```
sudo nano /etc/fail2ban/filter.d/juiceshop-login.conf
```

Add:

```
[Definition]
failregex = ^<HOST>.*POST /rest/user/login.* 401
ignoreregex =
```

Restart Fail2Ban:

```
sudo systemctl restart fail2ban
sudo fail2ban-client status juiceshop-login
```

```
Kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
hammad@Kali: ~
Session Actions Edit View Help
hammad@Kali: ~
$ sudo nano /etc/fail2ban/jail.local
(hammad@Kali)-[~]
$ cat /etc/fail2ban/jail.local
[juiceshop]
enabled = true
port = http,https
filter = juiceshop
logpath = /home/kali/juice-shop.log
maxretry = 5
bantime = 3600
findtime = 600
(hammad@Kali)-[~]
$ sudo nano /etc/fail2ban/filter.d/juiceshop.conf
(hammad@Kali)-[~]
$ cat /etc/fail2ban/filter.d/juiceshop.conf
[Definition]
failregex = .*POST /rest/user/login.* 401
ignoreregex =
(hammad@Kali)-[~]
```

Task 2

API Security Hardening

Rate Limiting

```
Kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
hammad@Kali ~
Session Actions Edit View Help
$ cat /etc/nginx/sites-available/juice-shop
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;

server {
    listen 80;

    location / {
        proxy_pass http://localhost:3000;
    }

    location /rest/user/login {
        limit_req zone=login_limit burst=3 nodelay;
        proxy_pass http://localhost:3000;
    }
}

(hammad@Kali)~$ sudo ln -s /etc/nginx/sites-available/juice-shop /etc/nginx/sites-enabled/

(hammad@Kali)~$ sudo systemctl restart nginx

(hammad@Kali)~$
```

CORS Configuration

```
File Machine View Input Devices Help
hammad@Kali ~
Session Actions Edit View Help
hammad@Kali: ~
GNU nano 9.0 /etc/nginx/sites-available/juice-shop *
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;

server {add_header Access-Control-Allow-Origin "http://localhost";
add_header Access-Control-Allow-Methods "GET, POST, PUT, DELETE";
add_header Access-Control-Allow-Headers "Content-Type, Authorization";
    listen 80;

    location / {
        proxy_pass http://localhost:3000;
    }

    location /rest/user/login {
        limit_req zone=login_limit burst=3 nodelay;
        proxy_pass http://localhost:3000;
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Install:

```
npm install cors
```

Add:

```
const cors = require('cors');

const allowedOrigins = [
  'https://yourdomain.com',
  'http://localhost:3000',
  'http://localhost:4200'
];

const corsOptions = {
  origin: function (origin, callback) {
    if (!origin) return callback(null, true);

    if (allowedOrigins.indexOf(origin) !== -1) {
      callback(null, true);
    } else {
      callback(new Error(`Origin ${origin} not allowed by CORS`));
    }
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-API-Key'],
  credentials: true,
  maxAge: 86400
};

app.use(cors(corsOptions));

app.use((err, req, res, next) => {
  if (err.message && err.message.includes('CORS')) {
    return res.status(403).json({
      error: 'CORS policy violation',
      message: 'Request origin is not authorized'
    });
  }
});
```

```
    next(err);
  });
```

API Key Authentication

```
const crypto = require('crypto');
const apiKeys = new Map();

function generateApiKey(userId) {
  const key = `juice_${crypto.randomBytes(32).toString('hex')}`;
  const hashedKey =
    crypto.createHash('sha256').update(key).digest('hex');

  apiKeys.set(hashedKey, {
    userId,
    created: new Date(),
    active: true
  });

  return key;
}

function apiKeyAuth(req, res, next) {
  const apiKey = req.header('X-API-Key');

  if (!apiKey) {
    return res.status(401).json({
      error: 'API key required'
    });
  }

  const hashedKey =
    crypto.createHash('sha256').update(apiKey).digest('hex');
  const keyData = apiKeys.get(hashedKey);

  if (!keyData || !keyData.active) {
```

```
    return res.status(403).json({
      error: 'Invalid API key'
    });
  }

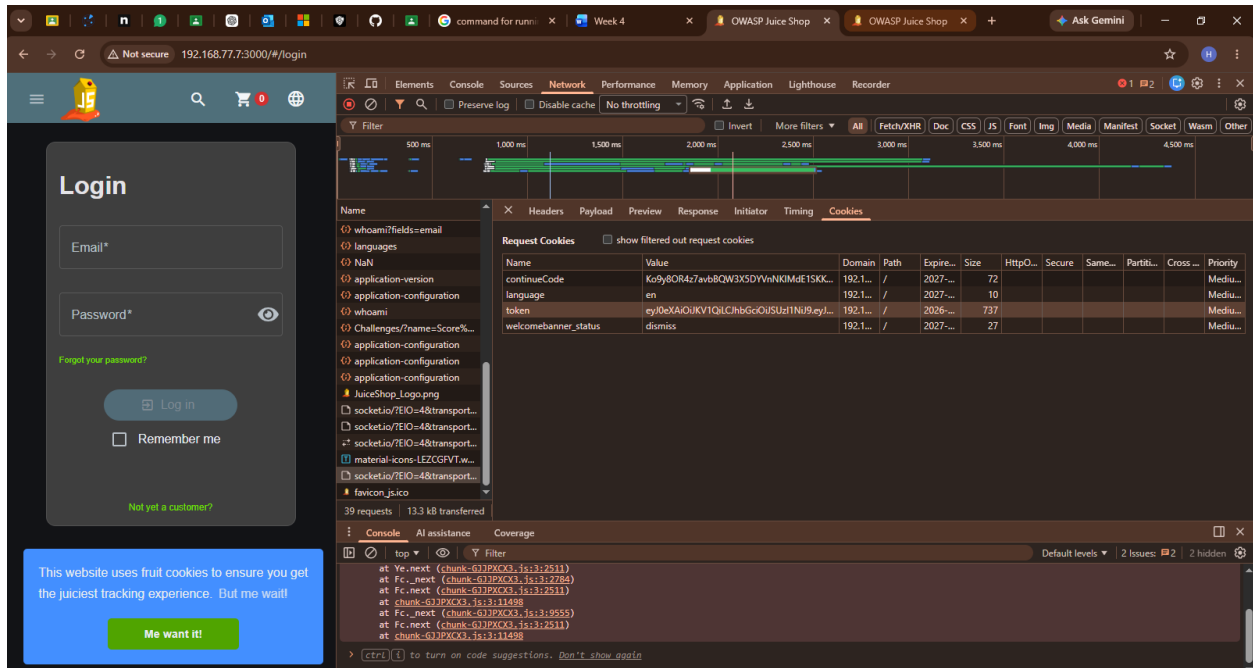
  req.apiUser = keyData;
  next();
}

app.use('/api/', apiKeyAuth);

app.post('/admin/api-keys', (req, res) => {
  const key = generateApiKey(req.user.id);

  res.json({
    apiKey: key,
    warning: 'Store this key securely. It will not be shown again.'
  });
});
```

JWT Authentication (Alternative)



```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

const JWT_SECRET = process.env.JWT_SECRET ||
  crypto.randomBytes(64).toString('hex');

app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;

  const user = await db.users.findByEmail(email);

  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const accessToken = jwt.sign(
    { userId: user.id, role: user.role },
    JWT_SECRET,
    { expiresIn: '1h' }
  );

  const refreshToken = jwt.sign(
    { userId: user.id, type: 'refresh' },
```



```
    JWT_SECRET,  
    { expiresIn: '7d' }  
  );  
  
  res.json({ accessToken, refreshToken, expiresIn: 3600 });  
});
```
